University of Dhaka
Department of Computer Science & Engineering
CSE 2206: Microcontroller and Embedded Systems

LAB 04

ADC Multi-Resolution Testing with Flash-Based Data Logging

Target Board: STM32 Nucleo-F446RE (STM32F446RE, RM0390)

1 Learning Outcomes – OBE Mapping

This lab is designed to build register-level skill in two peripherals – the ADC and embedded Flash – and, more importantly, in combining them to solve a real problem: giving sensor readings, test results, and student identity long-term memory on-chip.

CLO	Statement	Bloom's Level	Where Exercised
CLO3	Describe ADC (multi-resolution) and Flash peripheral architecture from RM0390	Understanding	Background theory, Milestone A
CLO4	Implement register-level firmware for ADC acquisition and Flash erase/program/read	Applying	Milestones A–E
CLO5	Analyse resolution trade-offs, Flash endurance, and failure modes	Analysing	Milestone F, Test Cases, Viva
CLO6	Design a persistent, multi-block Flash data-logging scheme (identity + test results)	Creating	Milestones C–D, Design Report

2 Assignment Objectives

By the end of this lab, you should be able to:

- Configure and read the STM32F446RE's on-chip ADC in polling mode, at multiple resolutions, from a potentiometer input.
- Perform Flash sector erase and word programming at the register level (`FLASH_KEYR`, `FLASH_CR`, `FLASH_SR`), including the unlock/lock sequence.
- Design a small non-volatile data layout across two Flash sectors: a write-once student identity block and a repeatedly-updated test-results block.
- Characterise how ADC resolution affects measurement precision, and relate each resolution to a realistic use case.
- Drive an automated, UART-triggered test sequence and persist its results so they survive resets and power cycles.
- Write and execute a structured test plan that distinguishes functional bugs from design-level weaknesses (e.g. Flash endurance).

3 Background

3.1 ADC Resolution and Precision

The STM32F446RE's SAR ADC can be configured for 12-, 10-, 8-, or 6-bit resolution (`ADC1_CR1.RES`). Regardless of resolution, voltage $=$ (code/max_code) $\times V_{REF+}$, where max_code is 4095, 1023, 255, or 63 respectively. Lower resolution gives fewer, coarser voltage steps (a larger 1-LSB step size) in exchange for a shorter conversion time – a genuine engineering trade-off, not just "worse." This lab has you characterise that trade-off directly rather than treat 12-bit as the only option worth using.

3.2 Why Store Data in Flash?

Two kinds of information in this lab need to survive a reset or power loss: your identity (written once) and your test results (rewritten every time you run the test suite). Flash is the only non-volatile storage on-chip, so both must live there rather than in RAM.

3.3 Flash Programming Constraints (RM0390 Ch. 3)

- Flash can only be programmed from 1 $\rightarrow$ 0 per bit. To change previously-written data, the containing sector must first be erased (which resets all bits to 1).
- Erasing is done per sector, not per byte – the STM32F446RE's 512 KB Flash is divided into 8 sectors of non-uniform size (4$\times$16 KB, 1$\times$64 KB, 3$\times$128 KB).
- Flash has a finite erase/program endurance (on the order of 10,000 cycles per sector) – a design implication you will be asked to reason about in Part 8.
- All Flash operations require unlocking the interface first (`FLASH_KEYR` key sequence) and should re-lock it afterward.

Design Decision Provided For You: Two Sectors, Two Purposes

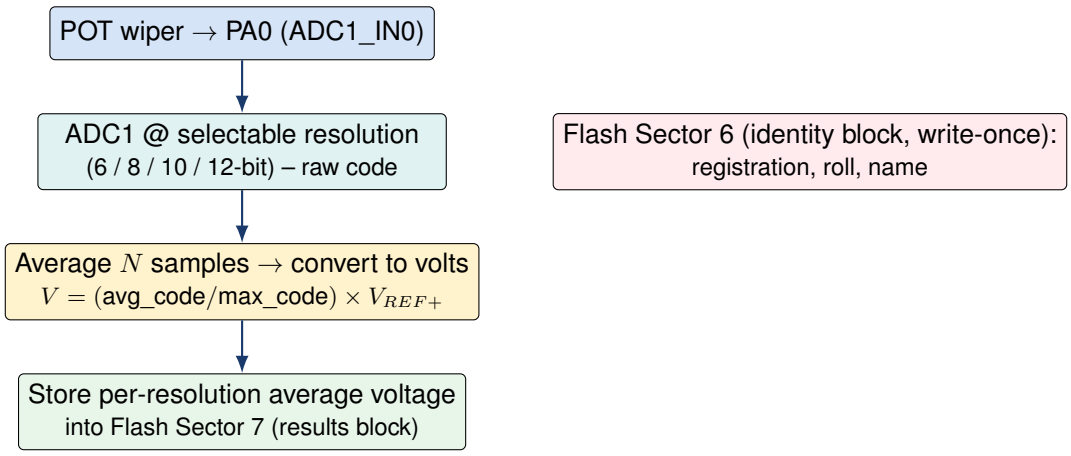
Because student identity is written once but test results are rewritten every run – and a sector erase wipes everything in that sector – this lab dedicates two separate Flash sectors: Sector 6 (base `0x08040000`) for your identity block (write once, never erased again), and Sector 7 (base `0x08060000`) for test results (erased and rewritten on every test run). This way, re-running tests can never accidentally destroy your stored identity. Confirm both addresses against the Flash sector table in RM0390 for your exact part before use.

4 Hardware Setup

Item	Purpose
STM32 Nucleo-F446RE board	Target MCU (STM32F446RE, Cortex-M4)
10 kΩ linear potentiometer	Analog input source – wiper to an ADC-capable pin (e.g. PA0)
Breadboard + jumper wires	Wiring the potentiometer's outer legs to 3V3/GND, wiper to PA0
USB cable + ST-Link (onboard)	Programming, power, and UART/debug output (no external measurement equipment required)

Wiring: connect the potentiometer's two outer terminals across 3V3 and GND, and its wiper (centre terminal) to PA0 (ADC1_IN0). Rotating the shaft sweeps the wiper voltage from 0 V to approximately 3.3 V. No multimeter or other external reference measurement is required for this lab – all voltage figures are derived directly from the known V_{REF+} and the ADC's own resolution, not from an externally measured ground truth.

5 System Design Overview



Boot sequence (every reset):

1. Display the identity block (Sector 6).
2. Display the previous test results, in volts, per resolution (Sector 7) – or "no data" if blank.
3. Wait for UART input → automatically run the test suite across all 4 resolutions → update Sector 7.

Student identity is written once (a provisioning step you run yourself, then never repeat) into Sector 6 and is never erased again by your application. The multi-resolution test suite is triggered by any UART input; it sweeps all four resolutions, averages several samples at each, converts each average to a voltage using that resolution's own max code, and stores all four results into Sector 7 – overwriting whatever was stored there from the previous run.

6 Implementation Milestones

Work through these in order – each builds on the previous and should be tested before moving on. Do not attempt Milestone C before A and B are independently verified working.

Milestone A – Raw ADC Acquisition

Configure ADC1 channel 0 (PA0) for single-conversion polling mode: analog GPIO mode, 12-bit resolution, appropriate sampling time, software-triggered start, poll EOC, read `ADC1->DR`. Print or otherwise display the raw code as you rotate the potentiometer.

Register-Level Implementation (Required)

```
// TODO: enable GPIOA + ADC1 clocks
// TODO: set PA0 to analog mode (MODER = 11)
// TODO: ADC1->SQR3 = 0;          // channel 0 first in sequence
// TODO: ADC1->SMPR2 |= (3<<0);   // reasonable sampling time for a POT
// TODO: ADC1->CR2 |= ADC_CR2_ADON;  // wake up, wait t_STAB

uint16_t ADC_ReadRaw(void) {
  // TODO: start conversion (SWSTART), poll EOC, return ADC1->DR
}
```

HAL-Based Equivalent (For Comparison)

Implement the same behaviour using STM32CubeIDE/HAL so you can compare the two approaches directly – note how much configuration detail HAL hides behind `ADC_HandleTypeDef` and `ADC_ChannelConfTypeDef`.

```
ADC_HandleTypeDef hadc1;

void ADC_Init_HAL(void) {
  // GPIO: PA0 as GPIO_MODE_ANALOG via HAL_GPIO_Init() (GPIOA clock enabled first)
  __HAL_RCC_ADC1_CLK_ENABLE();
```

```
  hadc1.Instance = ADC1;
  hadc1.Init.Resolution = ADC_RESOLUTION_12B;
  hadc1.Init.ScanConvMode = DISABLE;
  hadc1.Init.ContinuousConvMode = DISABLE;
  hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
  hadc1.Init.NbrOfConversion = 1;
  HAL_ADC_Init(&hadc1);

  ADC_ChannelConfTypeDef sConfig = {0};
  sConfig.Channel = ADC_CHANNEL_0;      // PA0
  sConfig.Rank = 1;
  sConfig.SamplingTime = ADC_SAMPLETIME_56CYCLES;
  HAL_ADC_ConfigChannel(&hadc1, &sConfig);
}

uint16_t ADC_ReadRaw_HAL(void) {
  HAL_ADC_Start(&hadc1);
  HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
  uint16_t raw = (uint16_t)HAL_ADC_GetValue(&hadc1);
  HAL_ADC_Stop(&hadc1);
  return raw;
}
```

Why Bother With Both?

The register-level version is what CLO3/CLO4 actually assess – it forces you to understand what ADON, SQR3, SMPR2 and EOC mean. The HAL version is what you are more likely to use in a real project. Building both here, on the same peripheral, lets you see exactly which register writes HAL is doing on your behalf.

Milestone B – Flash Utility Functions

Before touching identity storage or test logic, build and independently test low-level Flash functions, parameterised by sector number so the same code serves both Sector 6 (identity) and Sector 7 (test results):

Register-Level Implementation (Required)

```
void Flash_Unlock(void);    // write FLASH_KEY1 then FLASH_KEY2 to FLASH_KEYR
void Flash_Lock(void);      // set FLASH_CR.LOCK
void Flash_EraseSector(uint8_t sectorNum);
   // TODO: set FLASH_CR.SER, set SNB=sectorNum, set STRT, poll FLASH_SR.BSY
void Flash_WriteWord(uint32_t addr, uint32_t data);
   // TODO: set FLASH_CR.PG, write 32-bit data to addr, poll BSY, clear PG
uint32_t Flash_ReadWord(uint32_t addr);
   // Flash is memory-mapped -- a direct pointer dereference is sufficient
```

HAL-Based Equivalent (For Comparison)

HAL wraps the same unlock/erase/program/lock sequence behind `HAL_FLASH_*` and `HAL_FLASHEx_*` calls:

```
void Flash_EraseSector_HAL(uint32_t sectorMacro) {
  FLASH_EraseInitTypeDef eraseInit;
  uint32_t sectorError;

  HAL_FLASH_Unlock();
  eraseInit.TypeErase    = FLASH_TYPEERASE_SECTORS;
  eraseInit.Sector       = sectorMacro;   // e.g. FLASH_SECTOR_6 or FLASH_SECTOR_7
  eraseInit.NbSectors    = 1;
  eraseInit.VoltageRange = FLASH_VOLTAGE_RANGE_3;  // 2.7-3.6V, allows word programming
  HAL_FLASHEx_Erase(&eraseInit, &sectorError);
  HAL_FLASH_Lock();
}

void Flash_WriteWord_HAL(uint32_t addr, uint32_t data) {
  HAL_FLASH_Unlock();
  HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, addr, data);
  HAL_FLASH_Lock();
}
```

```
uint32_t Flash_ReadWord_HAL(uint32_t addr) {
  return *(volatile uint32_t *)addr;  // Flash is memory-mapped either way -- reads never
      need HAL
}
```

Checkpoint

Before Milestone C: erase Sector 7 (leave Sector 6 alone), write a known test pattern (e.g. `0xDEADBEEF`) to its base address, read it back, and confirm it matches. Also confirm that reading an erased (never-written) word returns `0xFFFFFFFF` – this is how you will detect "no data yet" at boot, for both the identity and results blocks. Do this for both your register-level and HAL implementations – they must agree.

Milestone C – Student Identification Storage

Design a write-once identity block, stored in Sector 6, containing your registration number, roll number, and name. A simple layout:

```
typedef struct {
  uint32_t marker;         // 0xB1010001 if valid, 0xFFFFFFFF if blank
  char registration[16];   // null-padded string
  char roll[12];           // null-padded string
  char name[32];           // null-padded string
} StudentInfo_t;            // 64 bytes total -- word-aligned
```

1. Write a one-time provisioning routine (run it yourself once, e.g. from a dedicated build/menu option – do not run it automatically every boot) that erases Sector 6 and writes your marker + details, one 32-bit word at a time.
2. At every subsequent boot, read the marker at the base of Sector 6. If it equals `0xB1010001`, print the registration number, roll number, and name. If it is blank (`0xFFFFFFFF`), print a clear "not yet provisioned" message instead of garbage.
3. Confirm that re-running Milestone D/E's tests (which only touch Sector 7) never disturbs this block.

Why a Struct, Not Just Numbers?

Registration number and roll number could be stored as integers, but name cannot – storing all three as fixed-size, null-padded character arrays keeps the layout simple and uniform, and every field is written the same way: as a sequence of 32-bit word writes.

Milestone D – Multi-Resolution ADC Test Suite

The ADC's resolution (`ADC1_CR1.RES`) is not just a precision dial – different resolutions suit different use cases. You will exercise all four and give each a concrete justification:

Resolution	RES bits	Max Code	Example Use Case
12-bit	00	4095	Precision sensing (e.g. fine-grained light or temperature monitoring)
10-bit	01	1023	General-purpose analog input (e.g. a volume/brightness dial)
8-bit	10	255	Coarse-but-fast input (e.g. simple threshold/level detection)
6-bit	11	63	Very coarse, high-speed sampling (e.g. a 3–4 position mode selector)

For each resolution, your test routine must:

1. Set `ADC1_CR1.RES` to the target resolution.

2. Take N samples (choose and justify N – e.g. 16) of the potentiometer and compute their average code.
3. Convert the average code to a voltage using that resolution's own max code: $V = \frac{\text{avg\_code}}{\text{max\_code}} \times V_{REF+}$.
4. Store all four resulting voltages (one per resolution) into the Sector 7 results block, alongside a validity marker (e.g. `0xCAFEBABE`).

UART Auto-Trigger (Required)

The full four-resolution test sequence must start automatically as soon as any byte is received on UART – poll `USART_SR.RXNE` (or use `HAL_UART_Receive` with a short timeout) in your main loop, and treat any received byte as "start test now," rather than requiring a specific command character. Debounce so a burst of bytes from one keypress does not restart the test sequence mid-run.

Milestone E – Boot Sequence & Result Display

At every reset, in this order:

- Display the identity block from Sector 6 (Milestone C).
- Read the Sector 7 results marker: if valid (`0xCAFEBABE`), display the four stored voltages, one per resolution, labelled clearly (e.g. "12-bit: 1.65 V"); if blank (`0xFFFFFFFF`), display "no previous test data."
- Enter the main loop and wait for UART input to (re-)run the Milestone D test suite, after which Sector 7 is updated with the new results.

Milestone F – Persistence & Robustness

Verify that both the identity block and the test results survive a software reset and a full power cycle (unplug/replug). Then complete the analysis questions in Section 8, which ask you to reason about Flash endurance and failure handling rather than just demonstrate happy-path behaviour.

7 Test Cases

Your submission must include a completed copy of this table with an actual/observed column and a pass/fail verdict for each row.

#	Test Case	Steps	Expected Result
TC1	Raw ADC responsiveness	Rotate POT fully min→max while printing raw code	Raw code changes monotonically, roughly 0 to ∼4095
TC2	Flash erase verification	Erase Sector 7, read base address	Read returns `0xFFFFFFFF`
TC3	Flash write/read-back	Write `0xDEADBEEF` to Sector 7 base, read back	Read returns exactly `0xDEADBEEF`
TC4	Identity provisioning	Run the one-time provisioning routine	Marker + registration + roll + name written; read-back matches what was entered
TC5	Identity display at boot	Reset the board after provisioning	Registration number, roll number, and name are displayed first, before anything else
TC6	Identity survives testing	Run the Milestone D test suite several times, then reset	Identity block is unchanged – only Sector 7 was touched
TC7	Blank results fallback	Erase Sector 7 only, reboot without running a test	System displays "no previous test data" and does not crash
TC8	Per-resolution averaging	Trigger a test run; log the raw samples at each resolution alongside the stored average	Stored average voltage matches an independently-computed average of the logged samples for every resolution
TC9	Result overwrite on re-test	Run the test suite twice via two separate UART triggers, with the POT moved between runs	Second run's four voltages are the ones shown after the next reset – no mixing of old/new data
TC10	Resolution boundary sanity	Inspect stored voltages at all four resolutions for the same POT position	All four are reasonably close to each other (within expected quantisation error), scaling with resolution
TC11	UART auto-trigger reliability	Send a single keypress, then a rapid burst of keypresses, on separate occasions	Both reliably start exactly one full 4-resolution test run – no missed trigger, no duplicate/overlapping runs
TC12	Power-cycle persistence	Unplug/replug USB after both provisioning and testing	Identity and last test results both survive and display correctly on the next boot

8 Analysis Questions (Answer in Your Report)

1. Why does Flash require an erase-before-write cycle, while RAM does not? Relate your answer to what a single Flash bit-cell physically represents.
2. Sector 7 has a finite number of erase cycles (≈10,000). If a user re-triggered the test suite every few seconds all day, estimate how long Sector 7 would last before likely failure. What design change would you make to reduce wear without losing the ability to see recent results?
3. Why does this lab store identity and test results in two separate sectors instead of one? What specifically would go wrong if both lived in the same sector?
4. Compare the four resolutions' stored voltages for the same POT position. Quantify the expected 1-LSB step size at each resolution and relate it to the differences you observed.
5. What is the risk of losing power in the middle of a Flash erase or write operation (e.g. during a test run), and how might a more robust design (briefly, conceptually) protect against a corrupted results block?
6. Compare your register-level and HAL implementations of `ADC_ReadRaw()`: what does HAL do for you that you had to do manually, and is there any timing/performance cost to that convenience?

9 Deliverables

- Complete, commented source code (`main.c` plus any supporting files), including both the register-level implementation (required) and the HAL-based equivalent (for comparison) for the ADC read and Flash utility functions. Source code is the primary artifact submitted for evaluation and marking.
- Completed test-case table (Section 7) with observed results and pass/fail.

- A short report (1–2 pages) answering Section 8 and briefly describing your test-trigger UX (how UART input starts a test run) and your identity-provisioning process.
- A photo or short video of your wired hardware setup.

10 Suggested Assessment Enhancement – Live Practical Component

Source-code submission alone cannot fully verify that a student can operate, explain, and adapt their own design under a live, slightly unpredictable situation – it mainly verifies that working code exists. To more directly test skill gain (not just deliverable completion), instructors are encouraged to add a short live practical component at grading/viva time:

- **On-the-spot parameter change**: ask the student to modify one small thing live – e.g. change the sample count N used for averaging, or add a 5th "resolution" entry to the results display – and rebuild/flash within a short time limit (e.g. 5–10 minutes). This distinguishes genuine understanding from copied working code far more reliably than reading the source alone.
- **Unplanned UART trigger timing**: have the instructor (not the student) decide when to send the UART trigger byte, including mid-boot or immediately after a reset, to check the design handles trigger timing robustly rather than only the sequence the student rehearsed.
- **Targeted viva question tied to the student's own code**: pick one specific line or design choice from their submission (e.g. "why did you erase Sector 7 here and not Sector 6?") and require a specific, on-the-spot explanation.
- **Basic originality check**: a simple diff/similarity check across submissions in the batch, flagging near-identical submissions for a closer look – cheap to run and a strong deterrent on its own.

Why This Matters

A student who only ever produces working code without being able to explain or adapt it under mild pressure has likely not internalised the skill the lab was designed to build. A 5–10 minute live component adds relatively little grading overhead but meaningfully raises confidence that the mark reflects the student's own understanding.

11 Assessment Rubric

Criterion	Weight	What is Assessed
Milestone A – ADC acquisition	10%	Correct register configuration; raw code responds correctly to POT
Milestone B – Flash utilities	15%	Correct unlock/erase/program/lock sequence; passes TC2–TC3
Milestone C – Identity storage & display	15%	Correct one-time provisioning; correct display order at boot; passes TC4–TC6
Milestone D – Multi-resolution test suite	15%	Correct RES configuration for all 4 resolutions; sound averaging; sensible use cases; passes TC8, TC10
Milestone E/F – Boot sequence, persistence & UART trigger	15%	Passes TC7, TC9, TC11, TC12
Test cases & report	15%	All 12 test cases executed and honestly reported; Section 8 answered with sound reasoning
Live practical component (Section 10, if used) / viva	15%	On-the-spot modification or explanation of own design choices

12 Submission Guidelines

- Submit a single zip archive: `<StudentID>_Lab04.zip` containing source code, report (PDF), and the media file. Source code is submitted specifically for evaluation and marking against Section 11.
- Late submissions follow the course's standard late-penalty policy unless a prior extension was granted.
- Code will be checked for originality; cite any external reference code you adapt, per the department's academic integrity policy.